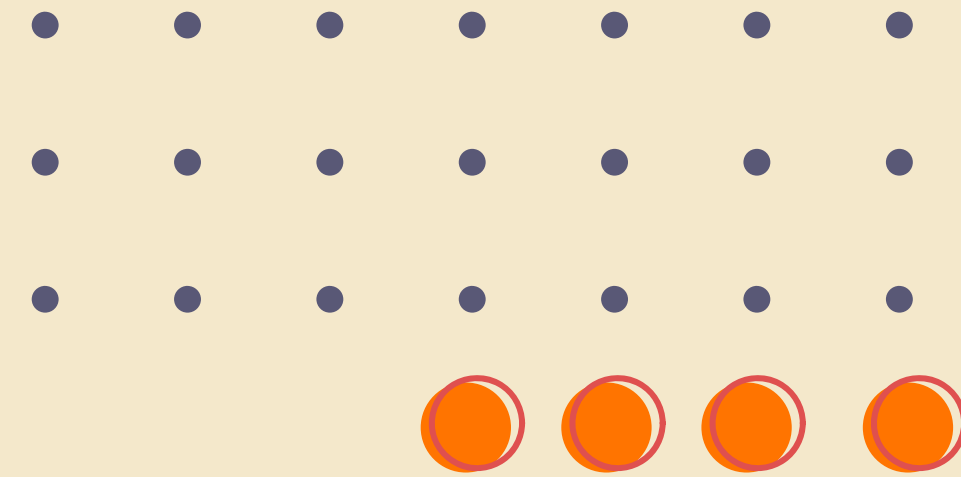
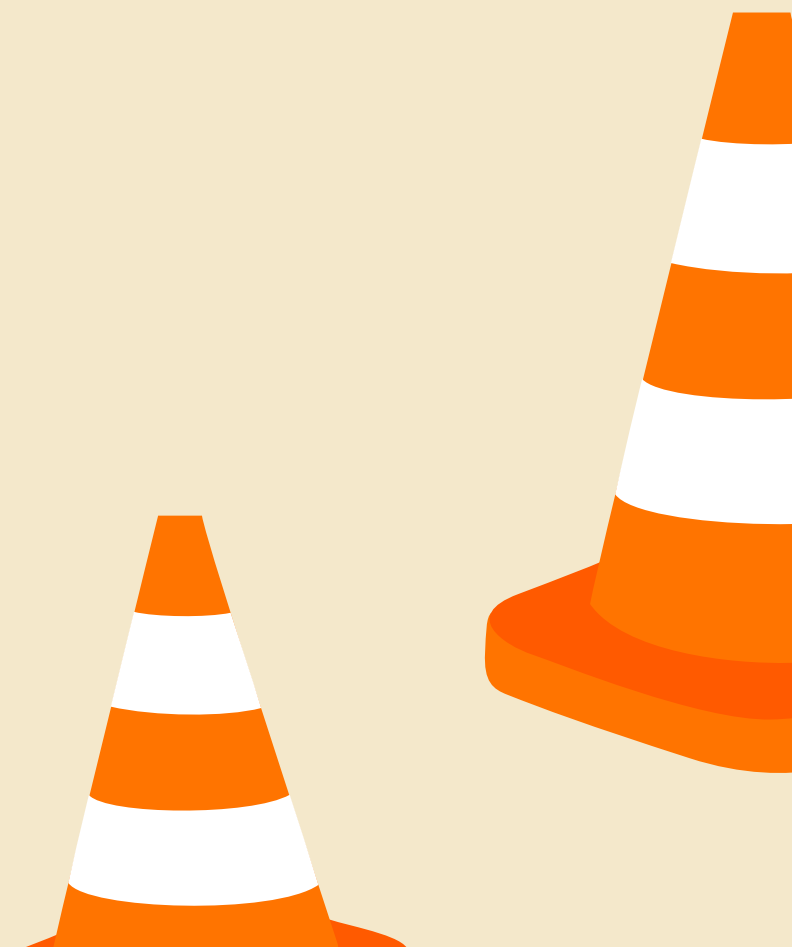




Motorcycle-Vehicle Interaction Safety Analysis Framework

Prepared by: Saurav Barua

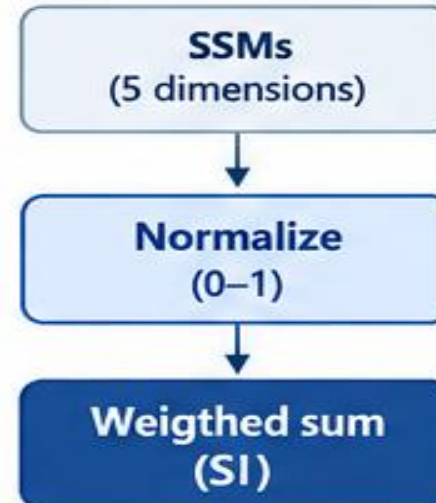


Methods: Six Safety Index (SI) Approaches

M1 – Weighted normalized

Uses literature-based weights for five conflict dimensions and computes a weighted sum of normalized SSMs.

	Temporal	0.35
	Lateral	0.25
	Longitudinal	0.15
	Relative speed	0.15
	Severity	0.10

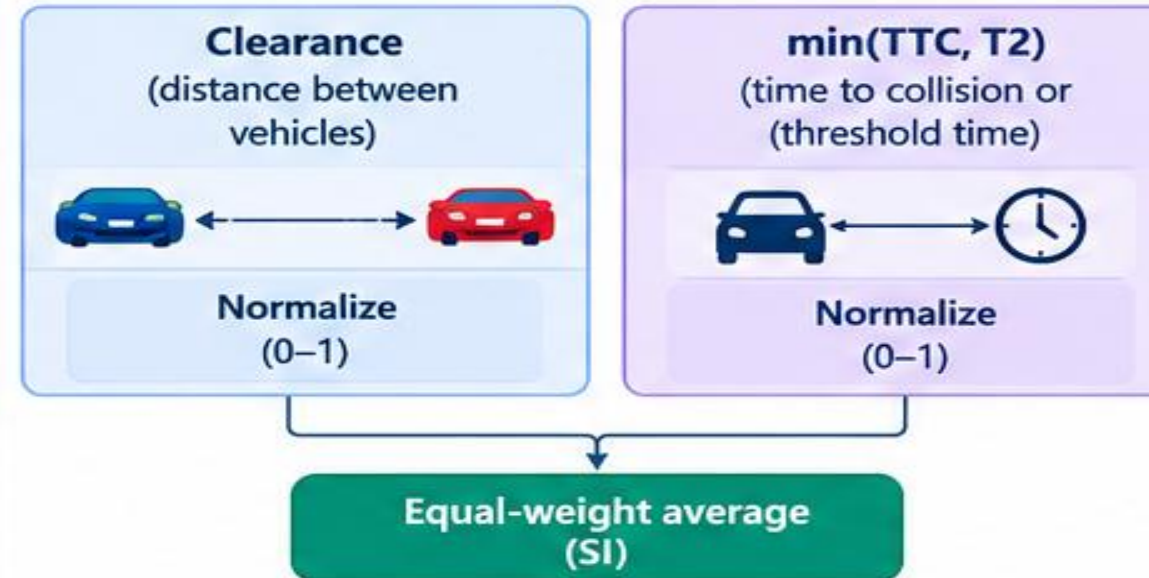


$$SI = 0.35 \cdot T + 0.25 \cdot L + 0.15 \cdot Lo + 0.15 \cdot RS + 0.10 \cdot Se$$

T = Temporal, L = Lateral, Lo = Longitudinal, RS = Relative speed, Se = Severity

M2 – Distance-time

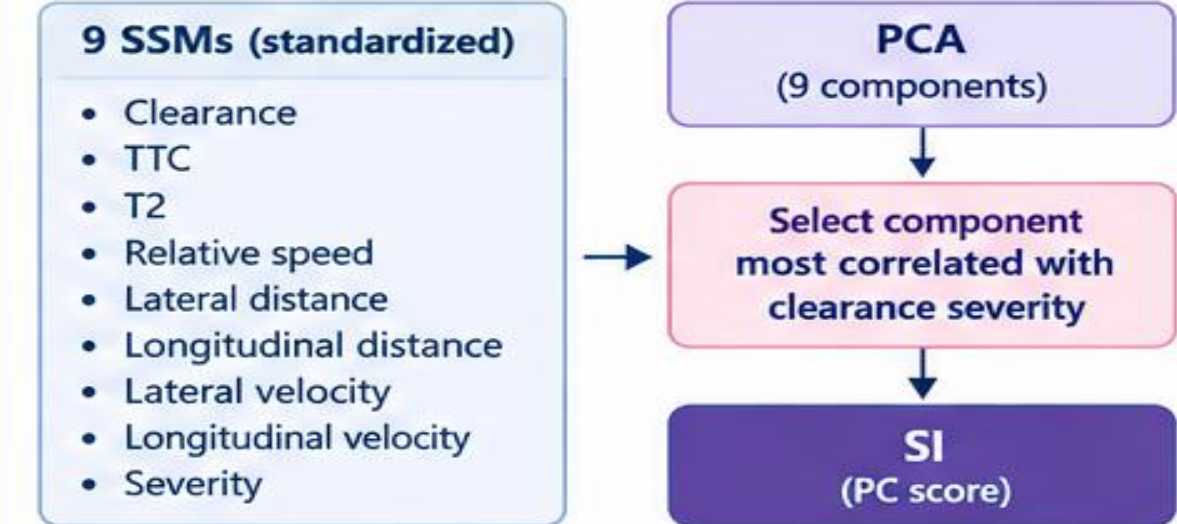
Uses an equal-weight average of normalized clearance and normalized min(TTC, T2).



$$SI = 0.5 \times \text{Clearance}_{\text{norm}} + 0.5 \times \text{min(TTC, T2)}_{\text{norm}}$$

M3 – PCA-based

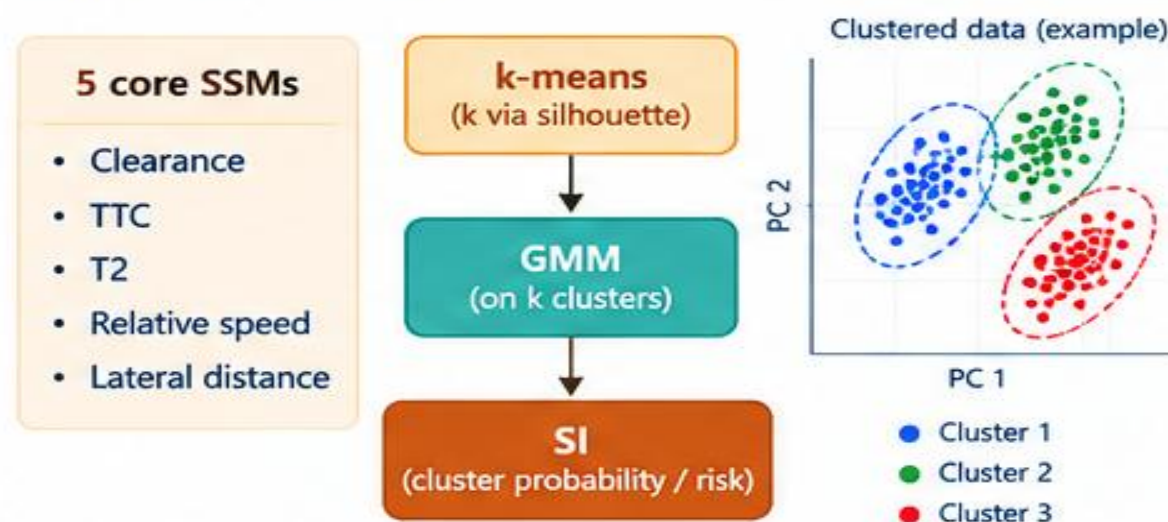
Fully data-driven approach. Performs PCA on 9 standardized SSMs and selects the component most correlated with clearance severity.



Data-driven | No manual weights | Uses all 9 SSMs

M4 – Clustering

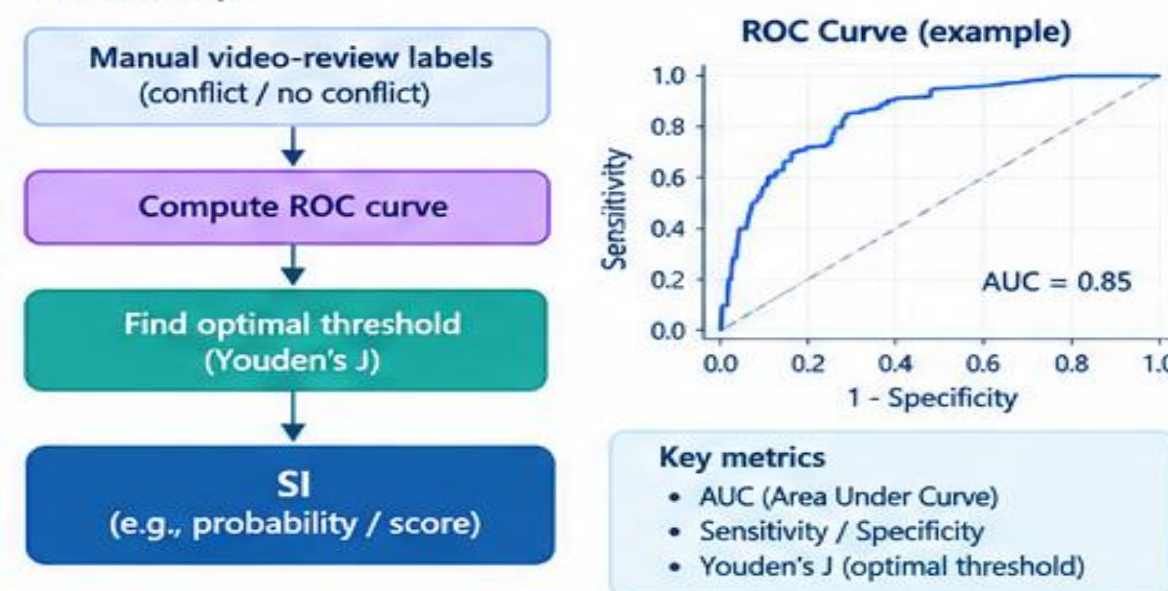
Uses k-means (k via silhouette) followed by GMM on 5 core SSMs. No labels needed.



Unsupervised | No labels needed | GMM for probabilistic clusters

M5 – ROC-based

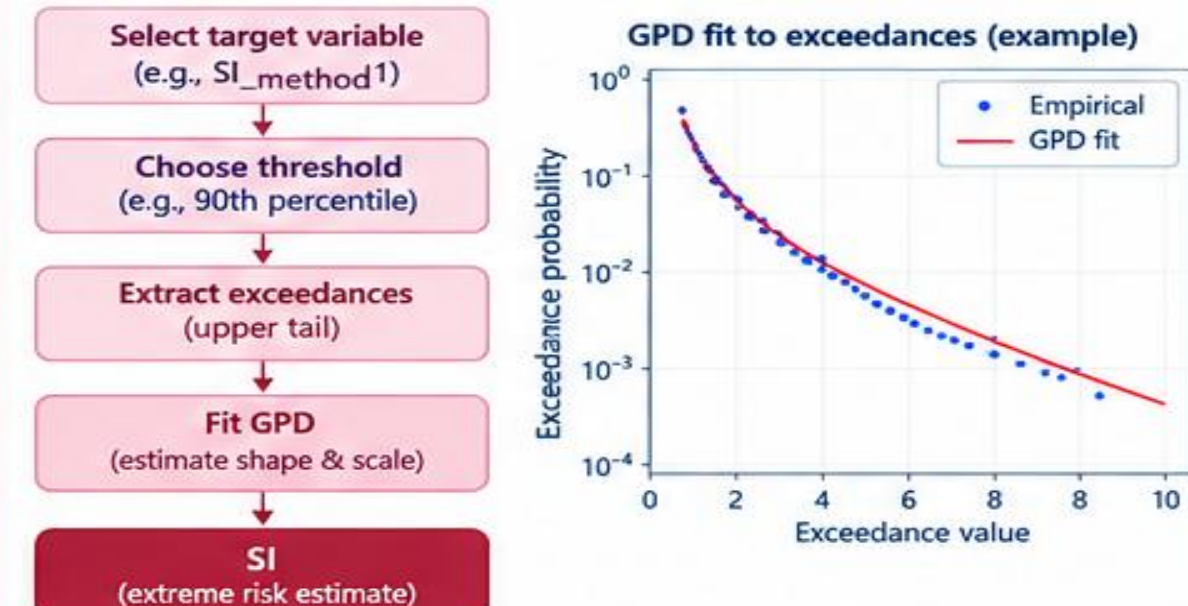
Uses manually labeled video-review data to evaluate classification performance (AUC, sensitivity, specificity, Youden's J).



Requires manual labels | Performance evaluation

M6 – POT-EVT

Fits a Generalized Pareto Distribution (GPD) to upper-tail exceedances — models rare/extreme conflicts.



Focuses on rare / extreme events | Better for tail risk estimation

Stage A — Identify candidate interaction pairs (pairing.py)

Goal: Find relevant motorcycle-vehicle (or vehicle-vehicle) pairs with temporal overlap and spatial proximity for further analysis.

1 Classify every track

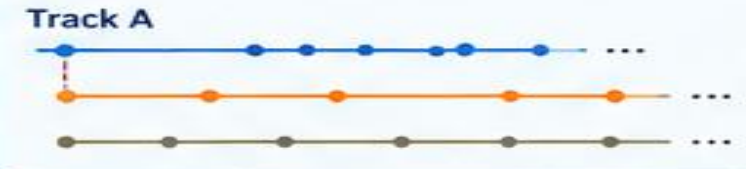
Map the type field to a class using keyword matching (config.py label lists).



2 Enumerate candidate pairs

For every pair of tracks (tid_a, tid_b) in a sheet, form combinations.

`itertools.combinations(track_ids, 2)`



All possible pairs are generated from the track IDs in the sheet.

3 Filter to relevant class combinations

If `require_motorcycle_in_pair = True` (default): keep pair only if:

- "Motorcycle" $\in$ {class_a, class_b}
- {class_a, class_b} $\in$ {MC-Car, MC-MediumVehicle, MC-Bus, MC-HeavyVehicle, MC-MC}

Example valid pairs



4 Check temporal overlap

$$t_{lo} = \max(\min_time_a, \min_time_b)$$
$$t_{hi} = \min(\max_time_a, \max_time_b)$$



Discard pair if $t_{hi} \leq t_{lo}$ (tracks never coexist).

5 Time-align the two tracks

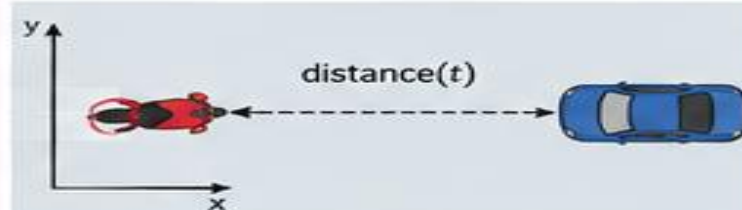
Merge using `merge_asof(..., direction="nearest")` with tolerance = $1.5 \times \max(\text{median } dt_A, \text{median } dt_B, 0.5s)$.



Match each frame of A to the nearest frame of B within tolerance.

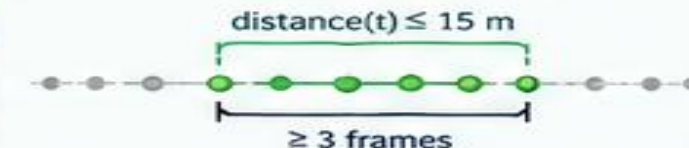
6 Compute instantaneous distance

$$\text{distance}(t) = \sqrt{(x_A(t) - x_B(t))^2 + (y_A(t) - y_B(t))^2}$$



7 Apply proximity + duration gate

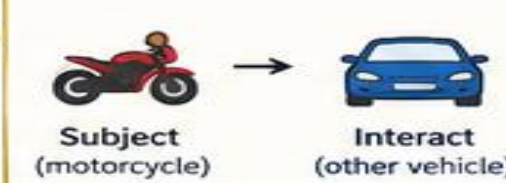
`close_frames` = frames where $\text{distance}(t) \leq 15 \text{ m}$
keep pair only if `count(close_frames)  $\geq 3$`



This is a screen, not a conflict classification; actual severity is judged later from SSM values.

8 Standardize roles

If a motorcycle is involved, relabel it as the "subject" vehicle (swap if needed).



9 Assign Event_ID

`Event_ID` = sheet_name + "_" + track_id_subject + "_" + track_id_interact

Output

A long-format "interaction-frame" table — one row per surviving (pair, timestamp).

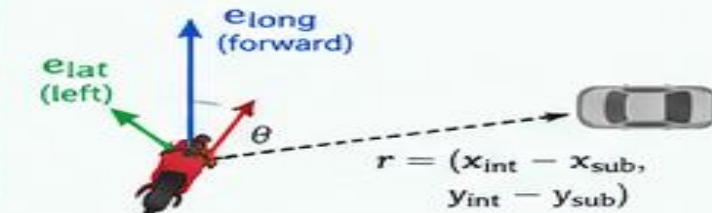
Stage B — Quantify the interaction geometry & collapse to one event (ssm.py)

Goal: Compute SSMs and event-level attributes for each pair, then reduce the pair trajectory to a single representative row.

1 Build motorcycle-centric frame

Use subject heading θ at each frame.

$$e_{\text{long}} = (\cos \theta, \sin \theta)$$
$$e_{\text{lat}} = (-\sin \theta, \cos \theta)$$



2 Longitudinal & lateral clearance

$$r = (x_{\text{int}} - x_{\text{sub}}, y_{\text{int}} - y_{\text{sub}})$$
$$C_x = r \cdot e_{\text{long}} \quad (\text{longitudinal})$$
$$C_y = r \cdot e_{\text{lat}} \quad (\text{lateral})$$
$$C_{2d} = \sqrt{C_x^2 + C_y^2}$$



3 Closing speed

$$v_{\text{rel}} = v_{\text{int}} - v_{\text{sub}}$$
$$\text{closing_speed} = -(v_{\text{rel}} \cdot \hat{r})$$

(positive = approaching)



4 Compute SSMs (per frame)

$$\text{TTC} = \text{distance} / \text{closing_speed}$$

if closing_speed > 0, else NaN

$$\text{T2} = \text{time to reach projected path-crossing point}$$

(NaN if near-parallel paths)

$$\text{PET} = |t_{\text{sub_conflict}} - t_{\text{int_conflict}}|$$

(conflict zone = radius = max_dist/3)

$$\text{DRAC} = (v_{\text{sub}} - v_{\text{int}})^2 / (2 \cdot \text{distance})$$

if closing & distance > 0

$$\text{Relative speed} = |v_{\text{int}} - v_{\text{sub}}|$$

5 Collapse to one event row

Use the frame of closest/most critical approach (e.g., minimum C_{2d}) to keep one row per pair.



6 Attach event-level summary attributes

- `peak_lateral_acc` = max(|lat accel of subject|)
- `peak_yaw_change` = max(|heading change of subject|)
- `heading_difference` = wrapped angular difference (subject vs. interacting vehicle at critical frame)

7 Log NaN notes

Whenever any SSM is NaN, add an explicit note (e.g., "separating, TTC undefined", "paths near-parallel, T2 undefined") in `SSM_notes` so downstream users know why the value is missing.



Examples:

- separating, TTC undefined
- paths near-parallel, T2 undefined

Net result

Each surviving (motorcycle, other-vehicle) pair becomes exactly one row in the event-level table, with:

- C_x, C_y, C_{2d}
- TTC, T2, PET, DRAC
- relative speed
- peak maneuver-severity fields

This table is the input to all six Safety Index methods.



Event-level table
(1 row per pair)

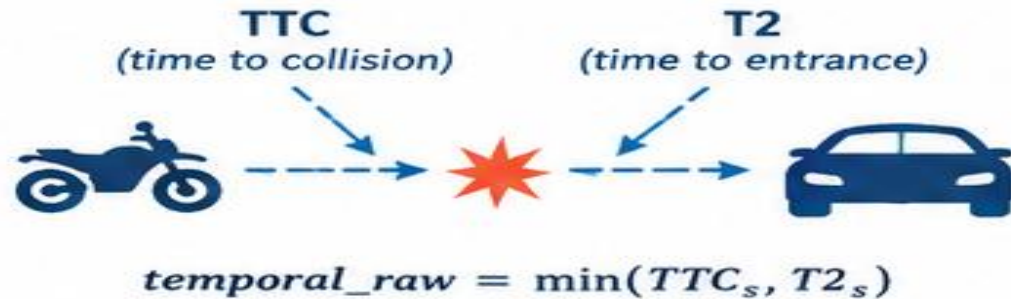
Method 1 — Weighted Normalized Safety Index

Combine five Surrogate Safety Measures (SSMs) into a single severity score (0–1) using normalization and weights.

1 Compute the temporal-proximity raw value

$$temporal_raw = \min(TTC_s, T2_s)$$

Use whichever conflict mechanism is more imminent. If one is missing, use the available one.



2 Normalize each raw SSM to a [0,1] severity scale

For “small = severe” measures

(temporal, lateral clearance, longitudinal clearance)

$lo = P1(\text{valid values})$, $hi = P99(\text{valid values})$
 $clipped = \text{clip}(x, lo, hi)$

$$severity = 1 - \frac{clipped - lo}{hi - lo}$$

- Temporal ($M1_temporal$)
- Lateral clearance ($M1_lateral$)
- Longitudinal clearance ($M1_longitudinal$)

For “large = severe” measures

(relative speed, |lateral acceleration|)

$lo = P1(\text{valid values})$, $hi = P99(\text{valid values})$
 $clipped = \text{clip}(x, lo, hi)$

$$severity = \frac{clipped - lo}{hi - lo}$$

- Relative speed ($M1_relspeed$)
- |Lateral acceleration| ($M1_severity$)

3 Apply literature-informed weights (from config.py)

Temporal ($w_temporal$) 0.35

Lateral ($w_lateral$) 0.25

Longitudinal ($w_longitudinal$) 0.15

Relative speed (w_rel_speed) 0.15

Severity ($w_severity$) 0.10

Weights sum to 1.00

4 Renormalize weights over only the non-missing components (for each event)

Example (one event)

Component	Raw value	Normalized severity	Weight	Contribution
Temporal	0.8	0.70	0.35	0.245
Lateral	1.2	0.60	0.25	0.150
Longitudinal	3.0	0.40	0.15	0.060
Rel. speed	5.5	0.30	0.15	0.045
Severity	2.0	0.20	0.10	0.020

Compute score using only available components

$$num = \sum_{i \in \text{available}} (w_i \cdot component_i)$$

$$denom = \sum_{i \in \text{available}} w_i$$

$$SI_{method1} = \frac{num}{denom}$$

If every component is missing
→ $SI_{method1} = NaN$

5 Output

$$SI_{method1} \in [0,1]$$

1 = most severe interaction

Retain all five component columns for auditability

$M1_temporal_component$

$M1_lateral_component$

$M1_longitudinal_component$

$M1_relspeed_component$

$M1_severity_component$

Key Points



1. Temporal raw
 $\min(TTC_s, T2_s)$
(uses available value)



2. Normalization
1st/99th percentile robust
min-max scaling



3. Weights
0.35 / 0.25 / 0.15 / 0.15 / 0.10
(from config.py)



4. Renormalization
over non-missing components
(per event)



5. Output
 $SI_{method1} \in [0,1]$
+ all 5 component columns

Method 2 — Distance-Time Safety Index

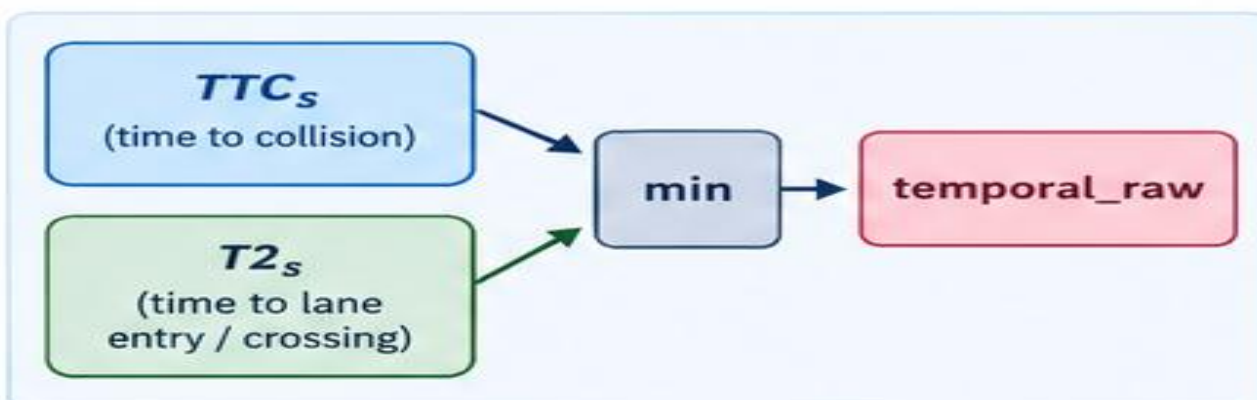
A simple, robust safety index using temporal proximity and 2-D clearance with equal weighting.

1 Compute Temporal-Proximity Raw Value

Use the more imminent conflict measure (skip NaN). Same as Method 1.

$$\text{temporal_raw} = \min(TTC_s, T2_s)$$

(row-wise, skipping NaN)



Result: temporal_raw (s) in seconds

2 Normalize Temporal Component

Use robust scaling (1st/99th percentile clip) across the whole dataset.

$$\begin{aligned} lo &= P1(\text{valid temporal_raw}) \\ hi &= P99(\text{valid temporal_raw}) \\ \text{clipped} &= \text{clip}(\text{temporal_raw}, lo, hi) \\ M2_temporal_component &= 1 - \frac{\text{clipped} - lo}{hi - lo} \end{aligned}$$

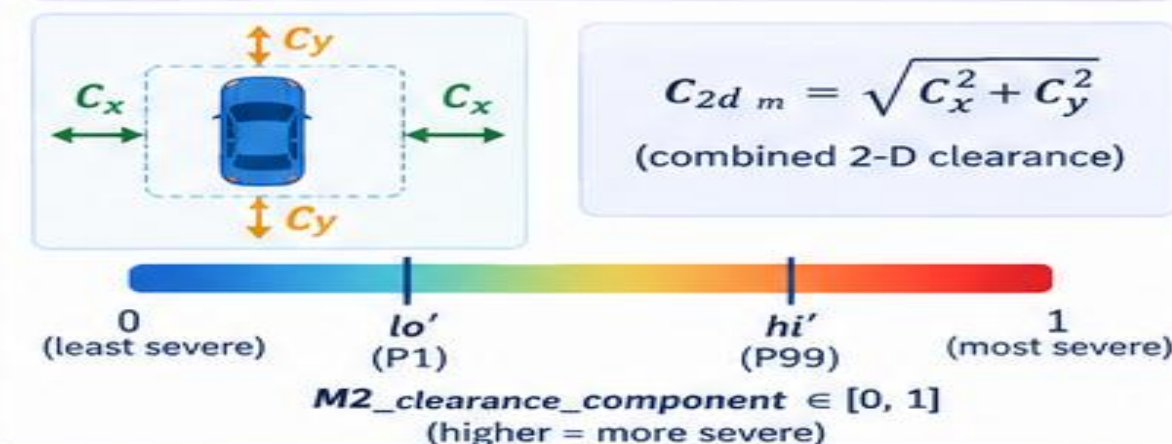


$M2_temporal_component \in [0, 1]$
(higher = more severe)

3 Normalize Clearance Component

Use combined 2-D clearance (not split into lateral/longitudinal).

$$\begin{aligned} C_{2d\ m} &= \sqrt{C_x^2 + C_y^2} \\ lo' &= P1(\text{valid } C_{2d\ m}) \\ hi' &= P99(\text{valid } C_{2d\ m}) \\ \text{clipped}' &= \text{clip}(C_{2d\ m}, lo', hi') \\ M2_clearance_component &= 1 - \frac{\text{clipped}' - lo'}{hi' - lo'} \end{aligned}$$



4 Combine with Equal Weights (50/50)

No analyst-chosen weighting scheme — equal weighting for a robustness check against Method 1.

$$\begin{aligned} num &= M2_temporal_component \cdot 1(\text{available}) \\ &\quad + M2_clearance_component \cdot 1(\text{available}) \\ denom &= 1(\text{temporal available}) + 1(\text{clearance available}) \end{aligned}$$

$$SI_{method2} = num / denom$$

(NaN if neither component is available)



5 Output

$SI_{method2} \in [0, 1]$, 1 = most severe.

Both component scores are retained for auditability.

Output Variables	Range
$SI_{method2}$	[0, 1]
$M2_temporal_component$	[0, 1]
$M2_clearance_component$	[0, 1]

Key Differences from Method 1



Uses combined 2-D clearance (C_x and C_y together)
Instead of separate lateral and longitudinal terms.



Equal weighting (50/50)
No analyst-chosen weights (robustness check against Method 1's weighted assumptions).



Same temporal normalization
(1st/99th percentile clip, small = severe).



Retains component scores
 $M2_temporal_component$ and $M2_clearance_component$ for auditability.

Method 3 — PCA-Based Safety Index

Step-by-Step

1 Select the feature set.

Nine SSM/kinematic variables per event:

TTC_s	T2_s	PET_s
C_x_m	C_y_m	C_2d_m
rel_speed_mps	peak_lateral_acc_mps2	
DRAC_mps2		

2 Sign-align time/clearance features so “large = severe” is consistent across all 9 variables.

Time and clearance measures are naturally “small = severe,” so they are negated before standardizing (relative speed, lateral accel., and DRAC are already “large = severe” and left unchanged):

```
for c in {TTC_s, T2_s, PET_s, C_x_m, C_y_m, C_2d_m}:  
    X[c] = -|X[c]|
```

Negate (time & clearance):

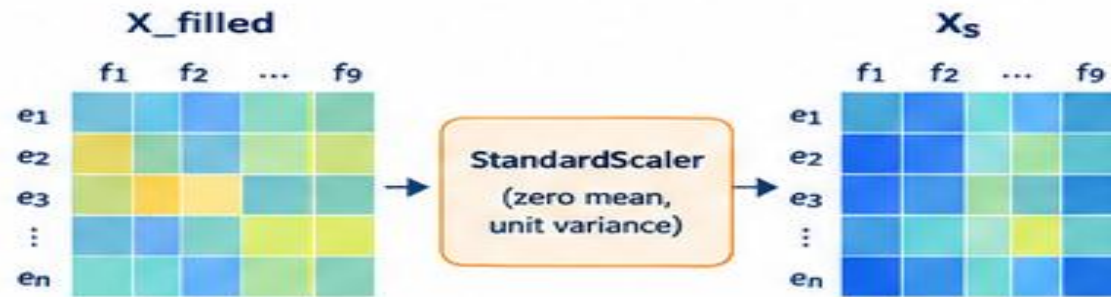
TTC_s, T2_s, PET_s,
C_x_m, C_y_m, C_2d_m

Keep as is:

rel_speed_mps,
peak_lateral_acc_mps2,
DRAC_mps2

4 Standardize the features (zero mean, unit variance).

So no single SSM dominates purely due to its raw scale.



5 Run PCA.

$n_{\text{components}} = \min(9, n_{\text{rows}}, n_{\text{features}})$

```
scores = PCA(n_components).fit_transform(Xs)  
# scores.shape = (n_events, n_components)
```

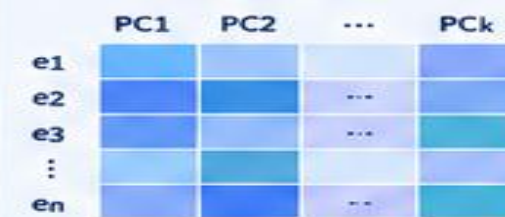
Standardized data (X_s)



n_events x 9

PCA

PC scores



n_events x n_components

3 Filter to usable rows and impute missing values.

- Keep only events with at least $\max(3, 9/2) = 5$ non-missing features (fewer → PCA is unreliable).
- If usable events < 5, PCA is skipped entirely and SI_method3 = NaN for all rows (logged in quality_log).

$X_{\text{valid}} = \text{rows with } \geq 5 \text{ non-missing features}$

$X_{\text{filled}} = X_{\text{valid}}$, missing cells filled with column median

$X_{\text{filled}} = X_{\text{filled}}$, any still-fully-missing column filled with 0

6 Select the principal component that best represents severity.

- For each component k , compute its correlation with the sign-aligned 2-D clearance:

$\text{corr}_k = \text{Pearson_corr}(\text{scores[:, } k], -C_{2d_m_aligned})$

- Choose the component with the largest absolute correlation:

$\text{best_pc} = \text{argmax}_k |\text{corr}_k|$

- If $\text{corr}_{\text{best_pc}} < 0$, flip the sign so larger values mean “more severe”:

$\text{pc_scores} = \text{scores[:, best_pc]}$ # or $-\text{scores[:, best_pc]}$

7 Rescale the chosen component to [0,1].

$$\text{SI_method3} = \frac{(\text{pc_scores} - \min(\text{pc_scores}))}{(\max(\text{pc_scores}) - \min(\text{pc_scores}))}$$

If all values are identical, defaults to 0.5 for every event.



8 Log diagnostics for transparency.

Write the following to PCA_Loadings / PCA_Explained_Variance:

- Which PC was chosen (best_pc)
- Its |correlation| with clearance severity
- Its explained-variance ratio
- Cumulative variance explained by the first 3 PCs

Example: PCA_Loadings

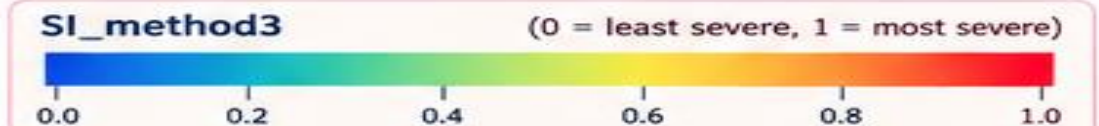
PC	correlation (with clearance)	Explained variance
PC3	0.78	0.12

Example: PCA_Explained_Variance

PC	Explained variance	Cumulative variance
PC1	0.34	0.34
PC2	0.21	0.55
PC3	0.12	0.67

9 Output.

$\text{SI_method3} \in [0,1]$, 1 = most severe.



- ✓ Only method with no analyst-chosen weights.
- ✓ Use as reference point to sanity-check Method 1's weight assumptions. (Compare SI_Method_Comparison_Correlation.csv)

Method 4 — Clustering-Based Safety Index

From SSM features to a discrete, severity-ordered safety index using K-means and GMM

1 Select the feature set

Use 5 core SSMs (reduced set for clustering stability).

CLUSTER_FEATURES

TTC_s	(time to collision)
T2_s	(time to second conflict)
C_2d_m	(2D clearance)
rel_speed_mps	(relative speed)
peak_lateral_acc_mps2	(peak lateral acceleration)

Compared to Method 3 (9 features), this smaller set improves clustering stability.

2 Sign-align time/clearance features

Make “large = severe” consistent across all features.

```
for c in {TTC_s, T2_s, C_2d_m}:  
    X[c] = -X[c]
```

Feature	Sign after alignment
TTC_s	(negated)
T2_s	(negated)
C_2d_m	(negated)
rel_speed_mps	(original)
peak_lateral_acc_mps2	(original)

Now, higher values always mean higher severity.

3 Filter to usable rows and impute missing values

- Keep events with ≥ 3 non-missing features (out of 5).
- Fill missing values with column median.
- If a column is fully missing, fill with 0.

Raw data
(5 features)

Filter
(≥ 3 non-missing)

Impute
(median / 0)

X_valid

! If total usable events < 6 , clustering is skipped entirely (SI_method4 = NaN for all rows, logged in quality_log).

5 Choose k via silhouette scan

Test $k = 2, 3, 4, 5$ and pick the best silhouette score.

```
for k in {2, 3, 4, 5}:  
    km_k = KMeans(n_clusters=k,  
                  n_init=10, random_state=42).fit(Xs)  
    silhouette[k] = silhouette_score(Xs,  
                                     km_k.labels_)  
k* = argmax_k silhouette[k]
```

Silhouette scores



If the scan fails, fall back to the config default $k = 3$.

6 Fit final clustering models

Compute both K-means and GMM on the full standardized feature set.

K-means

```
KMeans(n_clusters=k*,  
       n_init=10,  
       random_state=42)  
    .fit(Xs)
```

→ km_labels

GMM

```
GaussianMixture(  
    n_components=3,  
    random_state=42)  
    .fit(Xs).predict(Xs)
```

→ gmm_labels

SI_method4 is derived from the K-means result (GMM is reported for cross-checking).

7 Re-order cluster labels by severity

Use the mean of standardized features to order clusters (0 = least severe).

```
severity_score(event)  
= mean(Xs[event, :])
```

```
mean_severity(cluster)  
= average severity_score  
  of its members
```

Relabel clusters (ascending)
→ new labels 0 .. $k^* - 1$

Original labels
(arbitrary order)

2 0 1

Ordered labels
(by severity)

0 1 2

Apply the same relabeling to both K-means and GMM.

4 Standardize the features

Apply z-score standardization.

```
Xs = StandardScaler().  
    fit_transform(X_valid)
```

Feature	Before	After
TTC_s	-12.5	-1.32
T2_s	-8.7	-0.96
C_2d_m	-3.1	-0.45
rel_speed_mps	4.8	1.12
peak_lateral_acc_mps2	6.2	1.47

Standardized features have mean 0 and standard deviation 1.

8 Rescale K-means label to [0,1]

Convert the relabeled K-means cluster index to a safety index.

```
SI_method4 = SI_method4_cluster_kmeans /  
             (k* - 1) [if k* > 1, else / 1]
```

Example ($k^* = 3$):

Cluster (ordered)	SI_method4
0 (least severe)	0.00
1 (moderate)	0.50
2 (most severe)	1.00

SI_method4 $\in [0, 1]$ in discrete steps of $1/(k^* - 1)$.

9 Build the cluster summary table

For each ordered cluster, compute n_events and the mean of each feature.

Cluster (severity order)	n_events	TTC_s (mean)	T2_s (mean)	C_2d_m (mean)	rel_speed_mps (mean)	peak_lateral_acc_mps2 (mean)
0 (least severe)	245	-7.4	-6.8	-10.2	1.3	0.8
1 (moderate)	368	-3.2	-2.7	-5.4	3.6	2.1
2 (most severe)	142	-1.1	-0.8	-2.1	7.8	4.9

This table (Clustering.csv) helps interpret what distinguishes high-severity clusters from low-severity clusters (e.g., tight clearance + high lateral acceleration = evasive maneuvers).

10 Output

- ✓ SI_method4 $\in [0, 1]$ (discrete steps of $1/(k^* - 1)$).
- ✓ Both SI_method4_cluster_kmeans and SI_method4_cluster_gmm are retained.
- ✓ The two algorithms rarely disagree on the most-severe cluster but can differ on boundary cases.

Result: A stable, interpretable, and discrete clustering-based safety index.

Method 5 — ROC-Based Safety Index

Use manual labels to evaluate candidate safety indicators (SI_method1–SI_method4) with ROC analysis, select the best-performing method, and carry it forward as SI_method5 with an optimal threshold.

1 Load manual ground-truth labels

(required input; only method needing human review)



```
labels = read_manual_label_path  
(CSV/XLSX)
```

Find the first available label column:

```
{Final_Manual_Label, final_manual_label,  
label, conflict, Manual_Label_Reviewer1}
```



Require columns:

- Event_ID
- label_col



If no label file or usable label column is found, return immediately with a placeholder result: AUC = NaN, note = "no manual labels available". (This happened in the current run.)

2 Convert raw labels to binary via _to_binary()

Label (example values)	Binary
1, true, yes, conflict, critical, critical interaction, extreme conflict, evasive maneuver	1
0, false, no, non-conflict, normal, stable/normal, close interaction	0
Otherwise: • try float(label) > 0 → 1 • else 0 • unparseable → NaN (dropped)	

3 Merge labels onto event table



event_table
(with SI_method1–4)

+



labels
(binary)

```
merged = event_table.merge(labels,  
on="Event_ID", how="left")
```

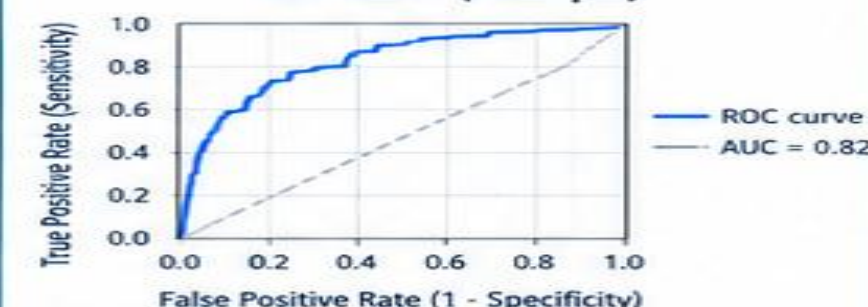
```
n_labeled = count(merged.manual_label_binary  
not NaN)
```

4 Compute ROC for each method

For each SI_method1 – SI_method4:

- 1 Use only rows where both the label and that SI score are available.
- 2 Skip method if <5 rows OR only one label class present.
- 3 `roc_curve(y_true, y_score)`
- 4 `AUC = roc_auc_score(y_true, y_score)`

ROC Curve (example)



5 Find optimal threshold (Youden's J)

For each method:

- 1 $J = tpr - fpr$
- 2 $best_i = \text{argmax}(J)$
- 3 $optimal_threshold = thr[best_i]$
- 4 $sensitivity_at_opt = tpr[best_i]$
- 5 $specificity_at_opt = 1 - fpr[best_i]$
- 6 $youden_J = J[best_i]$

Example result for one method

Threshold	Sensitivity	Specificity	Youden's J
0.42	0.78	0.85	0.63

6 Assemble results table

One row per candidate method (SI_method1 – SI_method4).

Method	AUC	Optimal Threshold	Sensitivity	Specificity	Youden's J
SI_method1	0.72	0.38	0.71	0.76	0.47
SI_method2	0.81	0.46	0.82	0.78	0.60
SI_method3	0.76	0.34	0.74	0.72	0.46
SI_method4	0.68	0.53	0.68	0.70	0.36

This table is saved as:



ROC_Results.csv

7 Select best method & create SI_method5

From the results table:

- 1 Keep rows with non-NaN AUC (valid methods).
- 2 $best_method = \text{row with highest AUC.}$



```
SI_method5 = event_table[best_method.method]
```

- SI_method5_source = best_method.method
- SI_method5_threshold = best_method.optimal_threshold

SI_method5 is not a new score — it is whichever of Methods 1–4 best discriminates true conflicts from non-conflicts against the manual labels, carried forward with its ROC-optimal threshold.

8 Output



ROC_Results.csv
(AUC, thresholds, sensitivity, specificity, Youden's J)

+



SI_method5
(from best-performing method)

- SI_method5_source
- SI_method5_threshold
- SI_method5 (scores per event)



If no method achieves a valid AUC (e.g., too few labels, single-class only), then SI_method5 = NaN for all events.



In short: Method 5 evaluates candidate safety indicators using manual labels and ROC analysis, selects the best-performing method, and outputs its scores as SI_method5 with an empirically derived threshold.

Method 6 — POT-EVT (Peaks-Over-Threshold Extreme Value) Index

Estimate rare-event (extreme) severity using the Generalized Pareto Distribution (GPD)

1 Select target variable and extract valid values

```
target_col = params.pot_target_variable
# default: SI_method1
raw = event_table[target_col].dropna()
```



If `len(raw) < 20`:

- Run proceeds, but is flagged as "illustrative diagnostics only, NOT reliable extreme-value estimates" in `quality_log`.



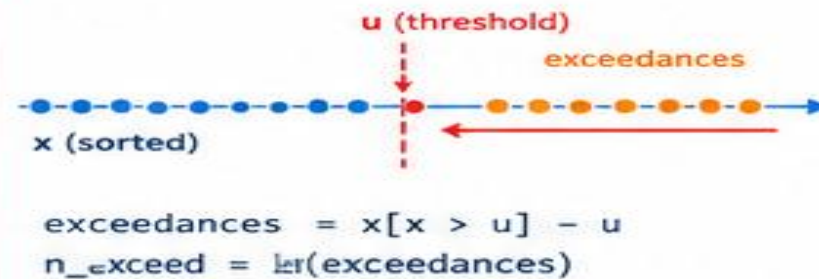
The target is assumed already on a "large = severe" scale (true for `SI_method1`). If using a raw clearance/TTC-type SSM where "small = severe," it would be negated automatically (not needed for default target).

```
x = raw.to_numpy()
```

2 Test multiple candidate thresholds

Use percentiles of `x`, not a fixed value.

Percentile (pct)	Threshold u
0.80	<code>quantile(x, 0.80)</code>
0.85	<code>quantile(x, 0.85)</code>
0.90	<code>quantile(x, 0.90)</code>
0.95	<code>quantile(x, 0.95)</code>



3 Fit GPD to exceedances (using MLE)

For each candidate threshold with ≥ 10 exceedances:

```
if n_exceed >= 10:
    (ξ, loc=0, σ) = genpareto.fit(
        exceedances, floc=0)
    record: ξ, σ, fit_status = "ok"
else:
    fit_status = "too few exceedances (<10) for stable MLE"
```

Output: `POT_EVT_Diagnostics.csv`

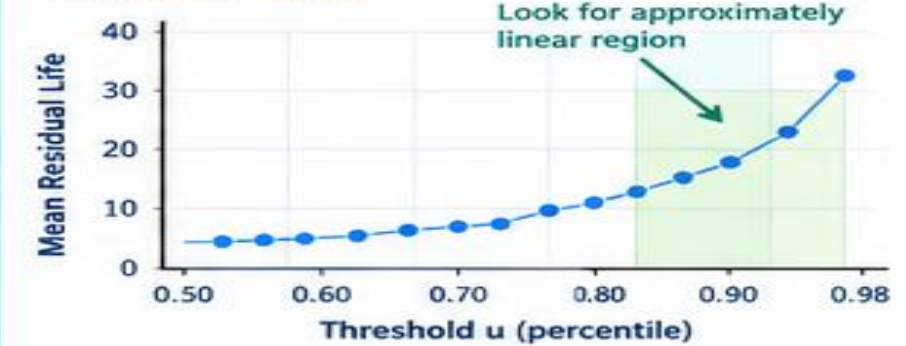
Percentile	u (threshold)	n_exceed	ξ (shape)	σ (scale)	Fit status
0.80	...	...	...	...	ok
0.85	...	...	...	...	ok
0.90	...	...	...	...	ok
0.95	...	...	...	...	too few

4 Mean Residual Life (MRL) diagnostic

Compute MRL over a finer grid of thresholds to check for a linear region.

```
grid = quantile(x, linspace(0.50, 0.98, 25))
for u in grid:
    exceed = x[x > u] - u
    MRL(u) = mean(exceed) if len(exceed) >= 5
            else NaN
```

MRL Plot (example)



5 Select final threshold

Use a transparent stability heuristic:

```
candidates_ok = diagnostics[diagnostics.fit_status == "ok"]
                    .sort_values("percentile")
chosen_pct = candidates_ok[0].percentile
(lowest percentile with a successful fit)
```

Logged explicitly:

- ✓ Chosen percentile (u)
- ✓ Number of exceedances (n_exceed)

6 Estimate return levels (rare-event severity)

For $N = \{10, 50, 100, 500\}$ (1-in- N events):

```
u, ξ, σ, n_exceed = fitted[chosen_pct]
n_total = len(x)
p_exceed_u = n_exceed / n_total
target_tail_p = 1 / (N * p_exceed_u)
if 0 < target_tail_p < 1:
    if ξ ≠ 0:
        level(N) = u + (σ/ξ) * (target_tail_p^(-ξ) - 1)
    else:
        level(N) = u - σ * ln(target_tail_p)
else:
    level(N) = NaN
```

Output: `POT_EVT_ReturnLevels.csv`

N (1-in- N)	Return level	u	ξ	σ
10	...	...	...	...
50	...	...	...	...
100	...	...	...	...
500	...	...	...	...

Only computed when $0 < \text{target_tail_p} < 1$.

Also includes (u , ξ , σ) as table attributes.

7 Output

GPD fit (at chosen threshold)

Parameter	Value (example)
ξ (shape)	0.18
σ (scale)	12.46
u (threshold)	42.7
n_exceed	28

Interpretation of ξ :

- $\xi < 0$ = bounded upper tail
- $\xi > 0$ = heavy / unbounded tail
- $\xi \approx 0$ = exponential tail

Return levels (rare-event severity)

N (1-in- N)	Estimated level (example)
10	78.4
50	112.7
100	138.9
500	215.6

Used to communicate rare/extreme-conflict risk rather than to rank typical events.

8 No event-level score assigned



POT-EVT does NOT assign a new `SI_method6` score to individual events.

Instead, it provides:

- ✓ GPD fit (ξ , σ) — tail shape
- ✓ Return-level estimates — expected severity of a 1-in- N event

9 Explicit limitation

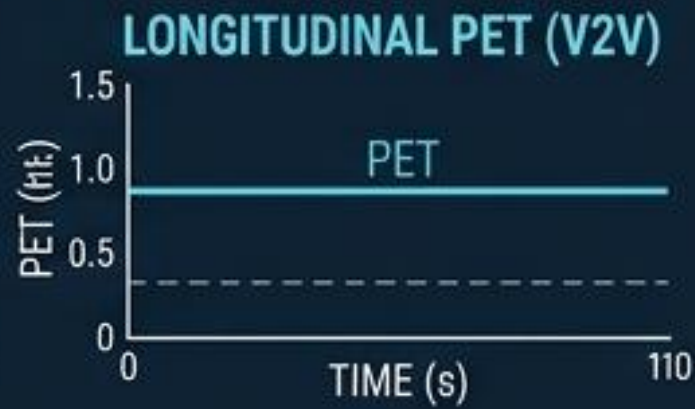
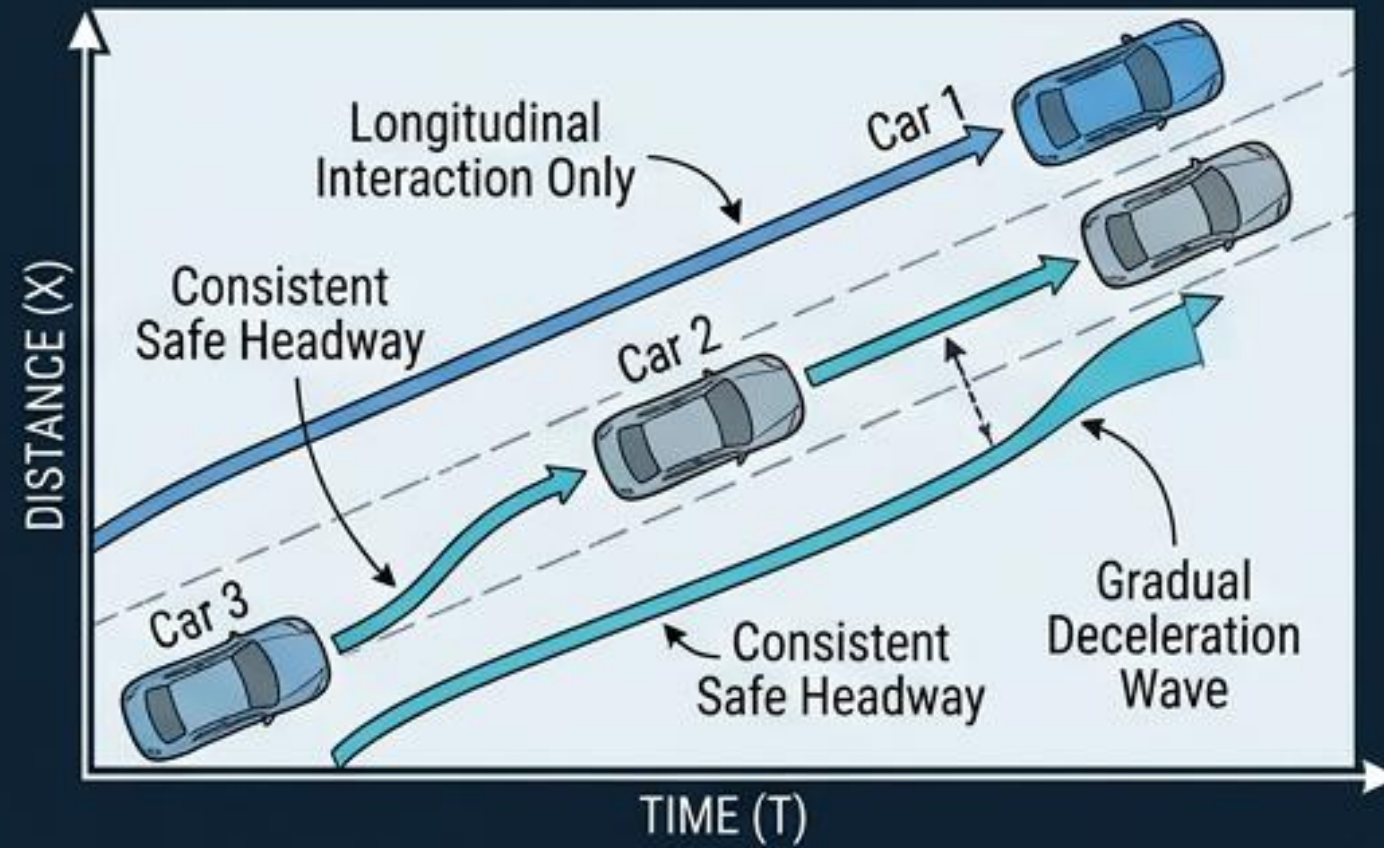


The module never asserts that any individual event is an "extreme conflict" purely for exceeding the EVT threshold.

Classification is deferred to `thresholds.py`, which combines POT-EVT output with the broader classification framework.

TIME-SPACE DIAGRAM: COMPARING VEHICLE INTERACTION TYPES & SURROGATE SAFETY MEASURES (SSMs)

VEHICLE-TO-VEHICLE (V2V) CAR-FOLLOWING CONFLICTS



TIME-SPACE DIAGRAM

Analyzing Vehicle-to-Vehicle (V2V) vs. Motorcycle-to-Vehicle (M2V) conflict dynamics using time-space trajectories and SSMs.

Vertical axis = **DISTANCE** (X, meters);
Horizontal axis = **TIME** (T, seconds).

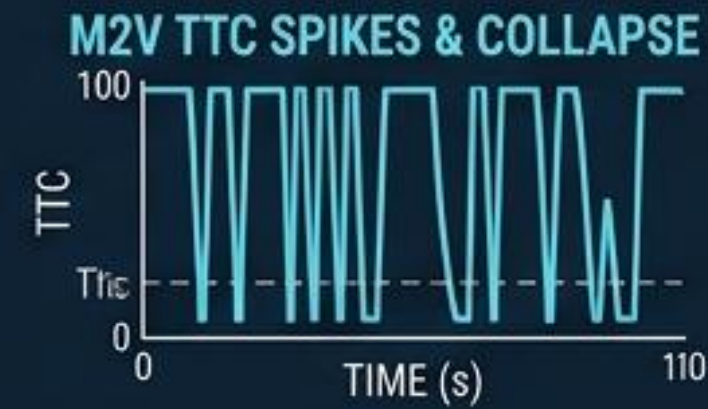
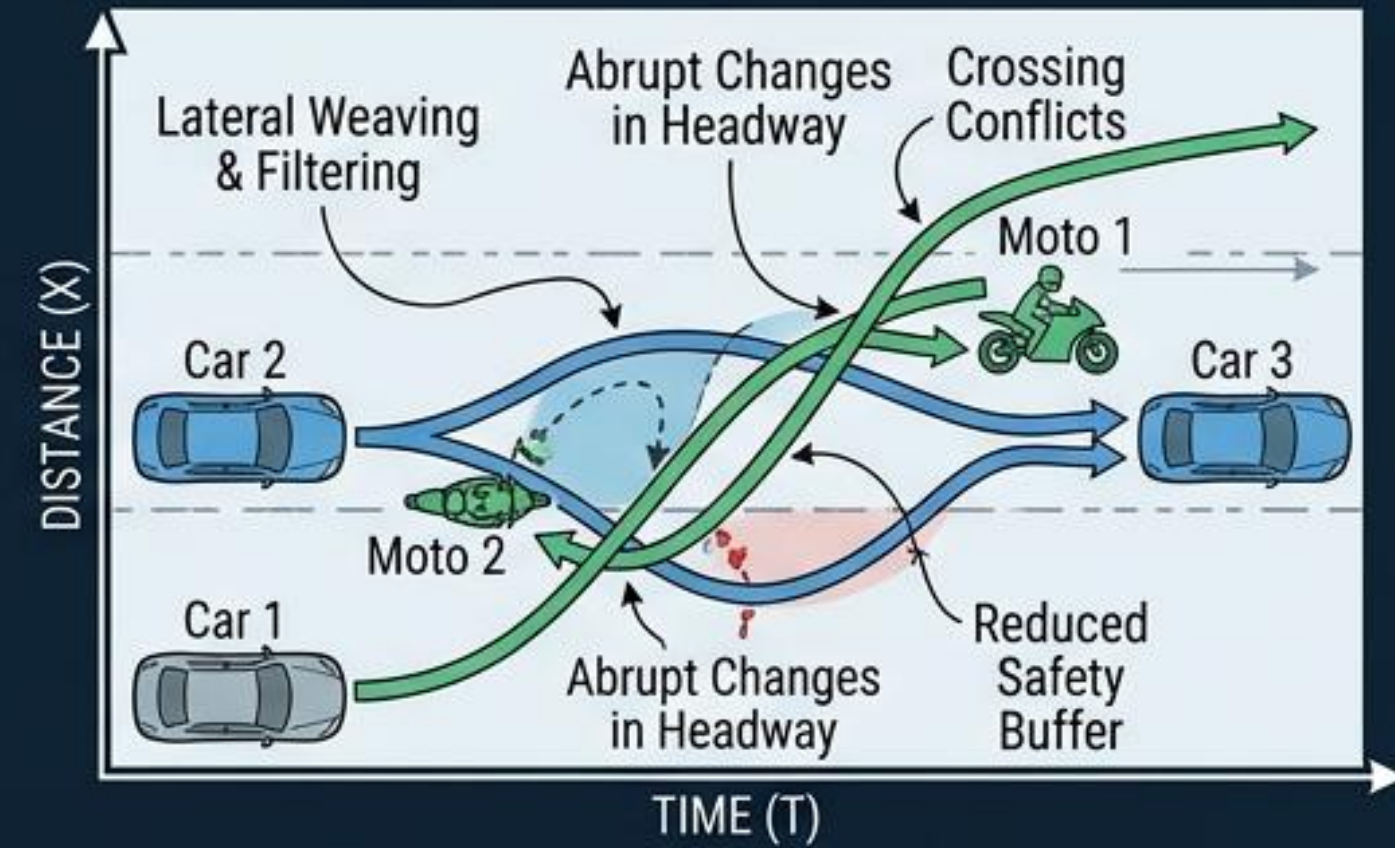


Collision Type:
Multi-Dimensional
Conflicts (Angle,
Lateral, Rear-End)



Conflict Zone:
involving lateral
displacement &
crossing points

MOTORCYCLE-TO-VEHICLE (M2V) LANE-SPLITTING & FILTERING CONFLICTS



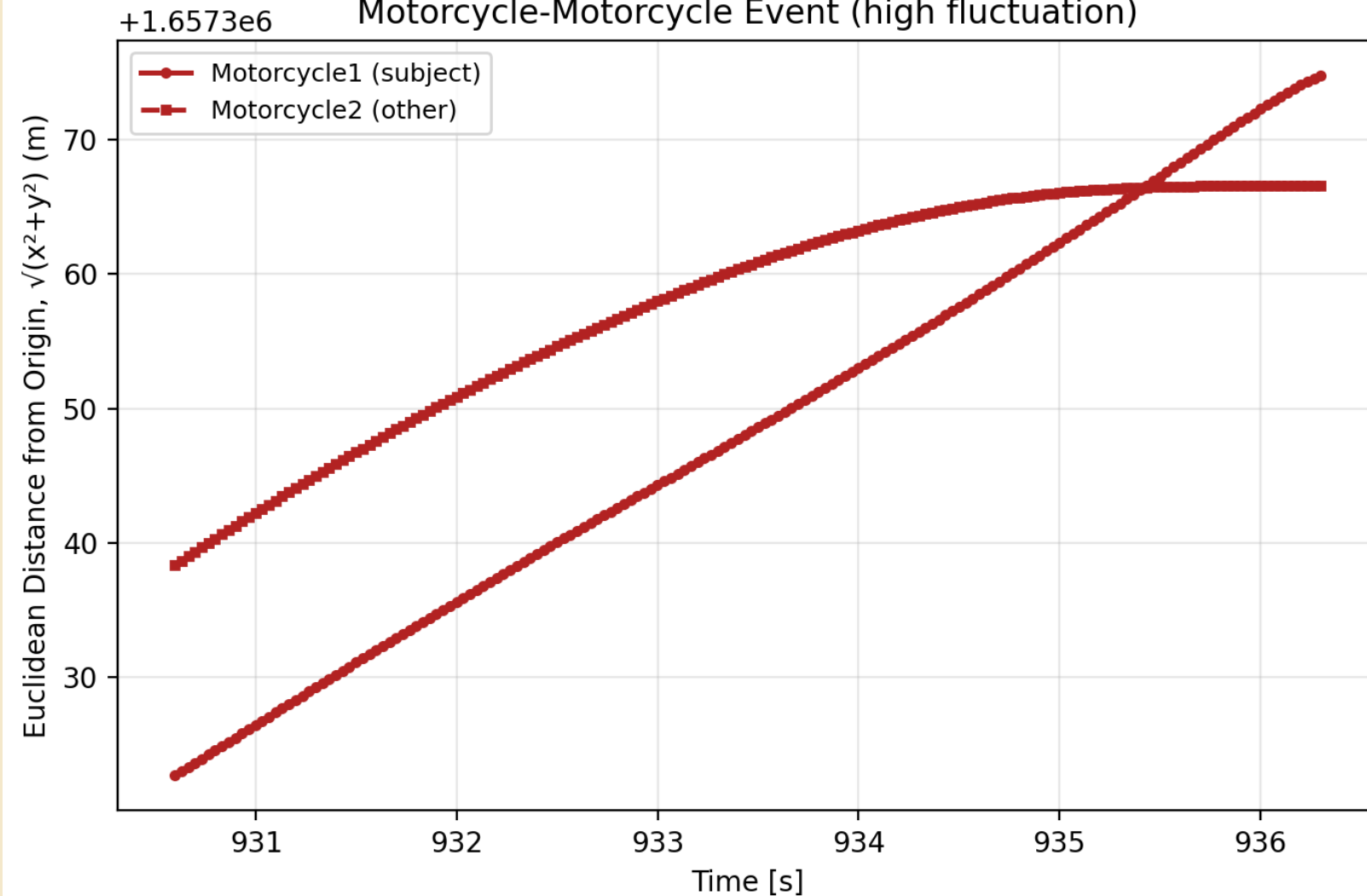
SUMMARY OF SSMs IN TIME-SPACE PLANE

	Parameter	V2V (Longitudinal)	M2V (Complex)
TTC (Time-to-Collision)	TTC (Time-to-Collision)	Smooth decrease, predictable	Sudden spikes, multi-dimensional
PET (Post-Encroachment Time)	PET (Post-Encroachment Time)	Moderate, queue tailbacks	Frequent near-zero events, lateral conflicts
Conflict Severity	Conflict Severity	Defined by speed difference	Defined by unexpected lateral movement & high deceleration

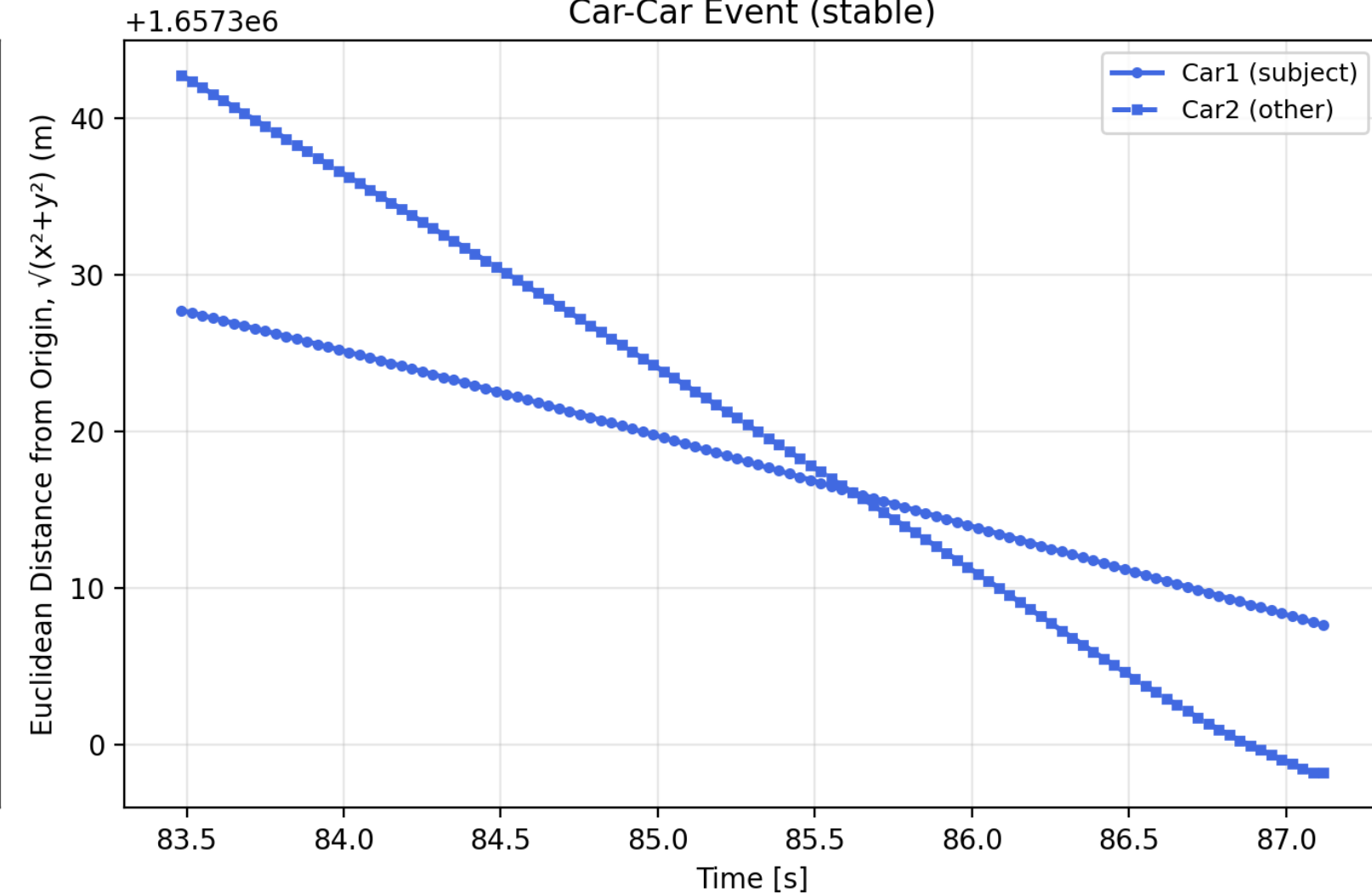
Test with real data (MC-MC, Car-Car interaction)

Space-Time Trajectories Extracted from 2_cross_test2_pivoted.xlsx (real vehicle tracks)

Space-Time Trajectory (Real Data): MC_MC_1
Motorcycle-Motorcycle Event (high fluctuation)



Space-Time Trajectory (Real Data): Car_Car_1
Car-Car Event (stable)

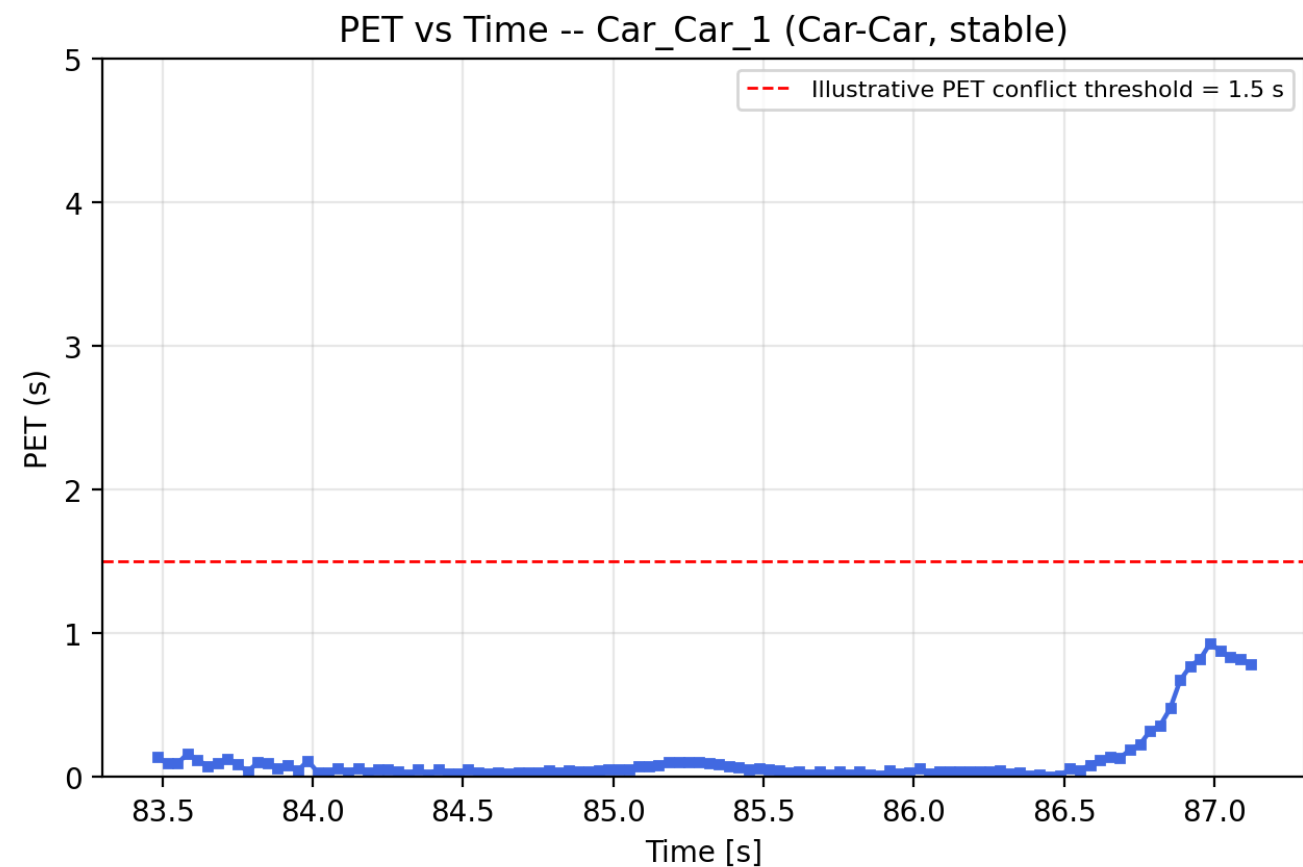
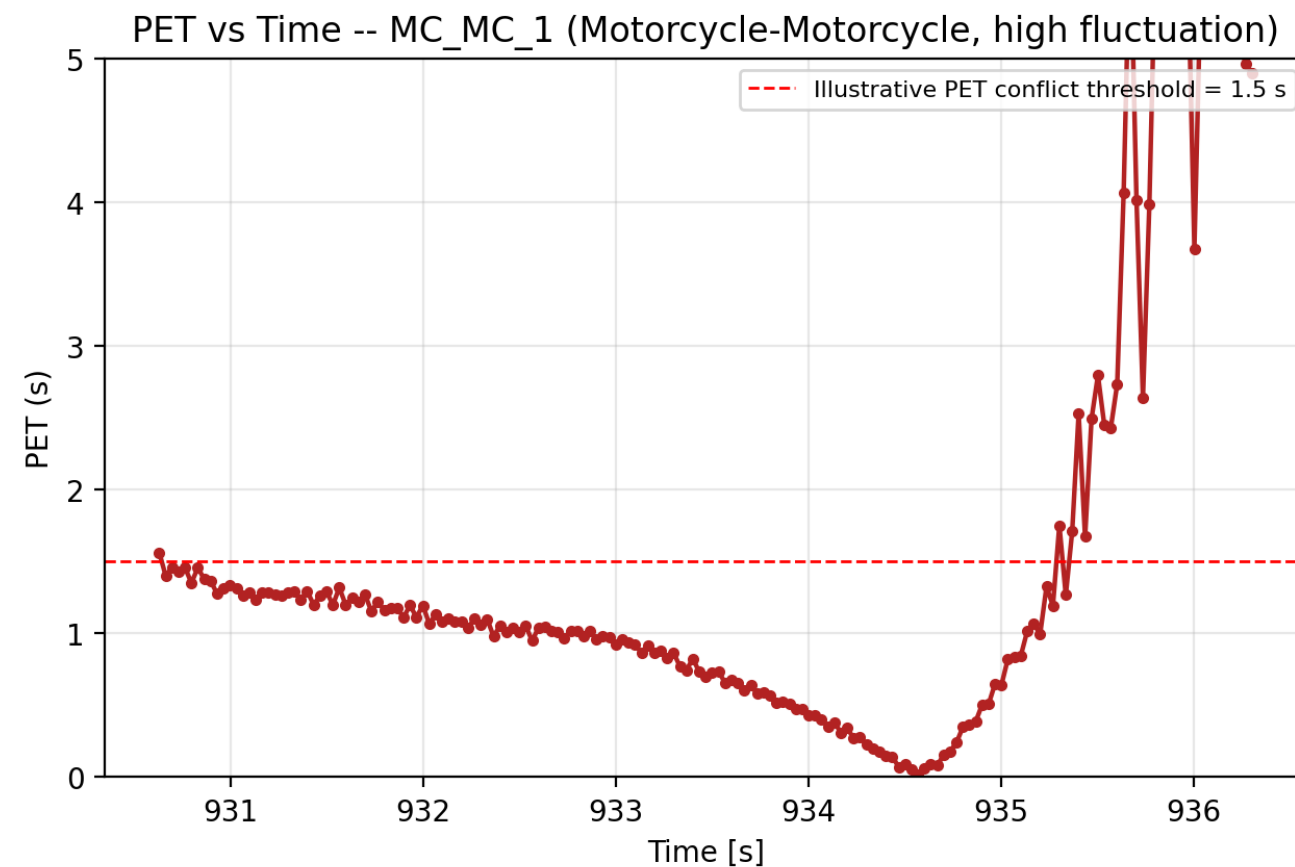
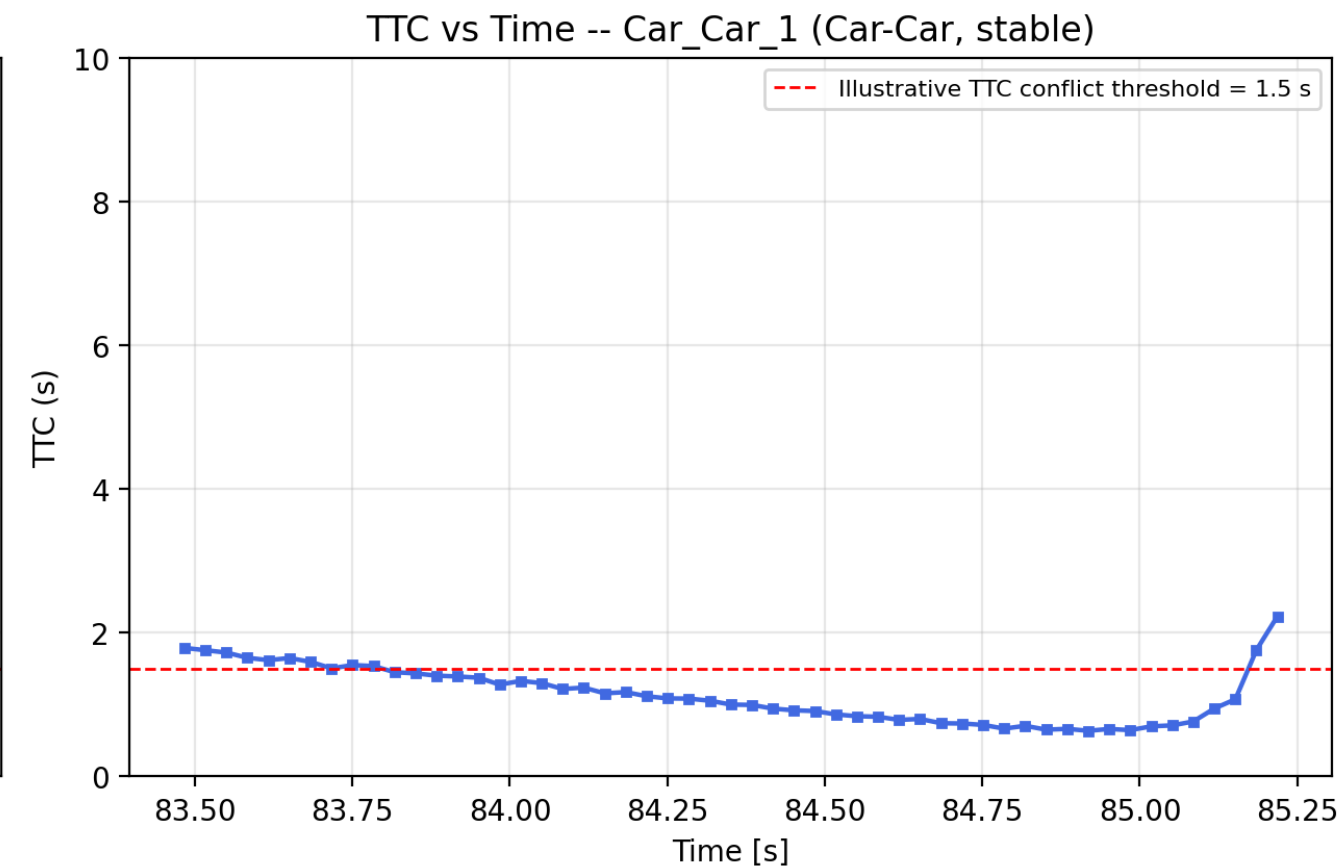
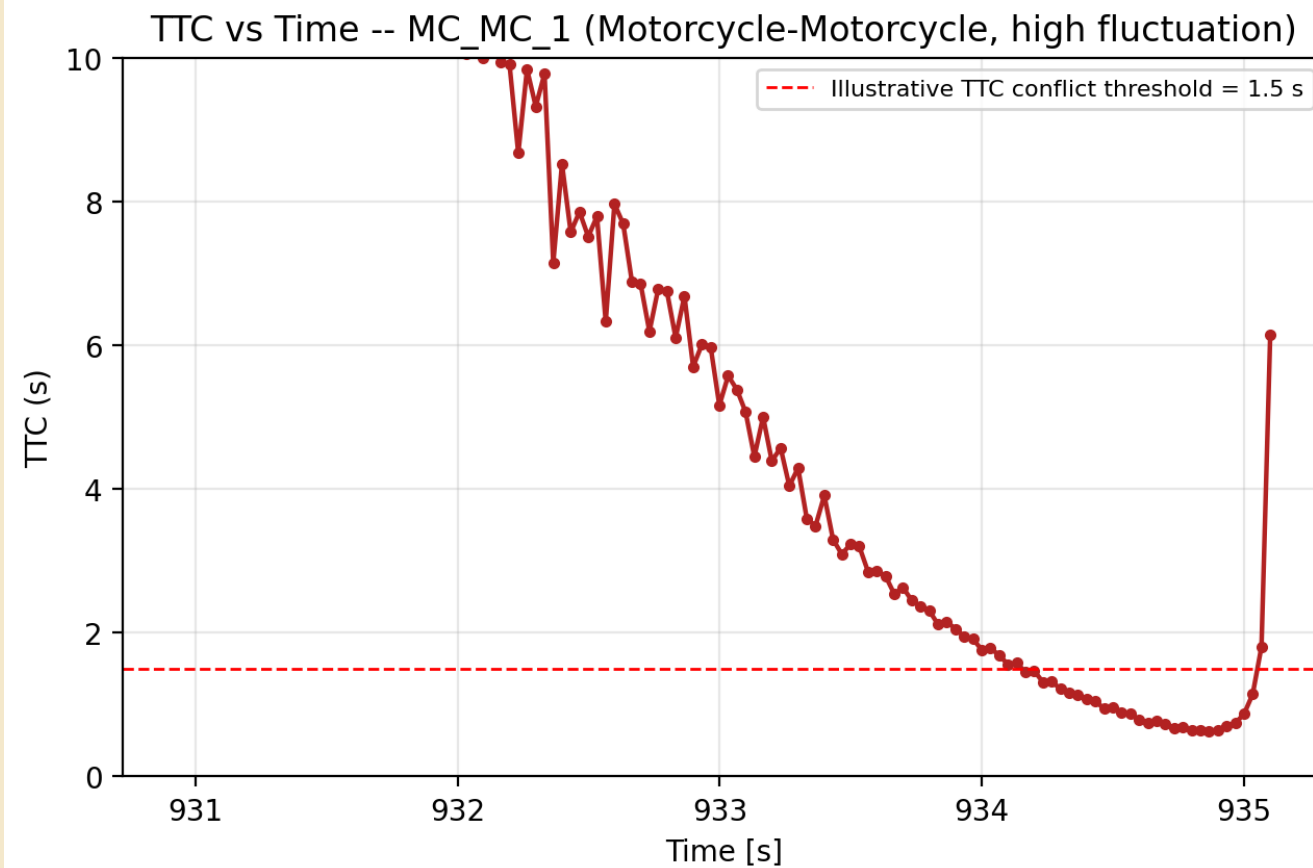


Two sample events:
1. MC-MC
2. Car-Car

Space-Time Trajectory Plot: The two motorcycles curve and cross paths, while the two cars move in smooth, straight lines toward each other.

Test with real data (MC-MC, Car-Car interaction)

TTC and PET Time Series (Real Data): Motorcycle-Motorcycle vs Car-Car Events



- **TTC vs. Time Plot:** Time-to-collision jumps around a lot for the motorcycles but drops smoothly and steadily for the cars.
- **PET vs. Time Plot:** The motorcycle event shows big ups and downs in PET, while the car event stays low and steady the whole time.

References

Reference	Method(s)	How it connects
Tarko, A.P. (2018). Estimating the expected number of crashes with traffic conflicts and the Lomax distribution. Accident Analysis & Prevention, 113, 63–73. DOI: 10.1016/j.aap.2018.01.008	Method 1, Method 6	Justifies weighting temporal SSMs highest (M1); establishes link between tail/extreme-value modeling of conflicts and crash risk (M6, GPD rationale)
Tarko, A.P. (2018). Surrogate measures of safety. In Safe Mobility: Challenges, Methodology and Solutions, Emerald Publishing, pp. 383–405.	Method 1	General SSM framework supporting temporal-proximity-first weighting hierarchy
Wang, C., Xie, Y., Huang, H., & Liu, P. (2021). A review of surrogate safety measures and their applications in CAV safety modeling. Accident Analysis & Prevention, 157, 106157. DOI: 10.1016/j.aap.2021.106157	Method 1, Method 3, Method 4	Reviews composite weighted SSM indices (M1); supports data-driven/dimensionality-reduction index construction (M3, PCA); supports clustering-based conflict classification without fixed thresholds (M4)
Lu, C., He, X., van Lint, H., Tu, H., Happee, R., & Wang, M. (2021). Performance evaluation of surrogate measures of safety with naturalistic driving data. Accident Analysis & Prevention, 162, 106403. DOI: 10.1016/j.aap.2021.106403	Method 1, Method 5	Empirically shows temporal (TTC-type) SSMs perform best on timeliness/robustness (M1 weighting rationale); demonstrates outcome-based SSM performance evaluation (M5, ROC/AUC logic)
De Ceunynck, T., et al. (2018). In search of surrogate safety indicators for vulnerable road users. Transport Reviews. DOI: 10.1080/01441647.2018.1442888	Method 1	VRU-specific (motorcyclist-relevant) support for combining temporal + proximity indicators

References

Reference	Method(s)	How it connects
Calibration and validation of a new time-based surrogate safety measure using fuzzy inference system. IATSS Research, 2016. DOI: 10.1016/j.iatssr.2015.11.001	Method 2	Precedent for a "mixed index" combining time-based (TTC/PET) and distance-based measures with equal/no differential weighting
Mahmud, S.M.S., Ferreira, L., Hoque, M.S., & Tavassoli, A. (2017). Application of proximal surrogate indicators for safety evaluation. IATSS Research, 41(4), 153–163. DOI: 10.1016/j.iatssr.2017.02.001	Method 4	Supports letting statistical structure in SSM data (not fixed thresholds) define conflict severity groups — basis for clustering approach
Songchitruksa, P., & Tarko, A.P. (2006). The extreme value theory approach to safety estimation. Accident Analysis & Prevention, 38(4), 811–822. DOI: 10.1016/j.aap.2006.02.003	Method 5, Method 6	Early precedent for validating SSMs against observed outcomes (M5, ROC logic); foundational EVT/POT application to conflict severity (M6)
Maji, A., & Ghosh, I. Analyzing Roundabout Safety under Mixed Traffic: A Bivariate EVT-POT Approach.	Method 6	Direct analogue — applies POT-EVT to TTC/speed-differential SSMs in mixed motorcycle-heavy traffic

Note: No reference specifies the exact numeric weights (0.35/0.25/0.15/0.15/0.10) used in Method 1 — these remain the framework's illustrative default. Methods 2, 3, and 4 use no literature-derived weights at all (equal-weight, PCA-derived, and cluster-derived respectively), so their references support the design approach, not specific coefficients.